

Pattern Recognition — Complete Suggestion Question Solutions

Q1. (a) Feature Selection and Its Importance

Feature Selection is the process of choosing a subset of the most relevant, informative, and non-redundant features from the full feature set, while discarding irrelevant or redundant ones — without transforming the original features.

Why it is important in Pattern Recognition:

- 1. **Reduces dimensionality:** Fewer features → less computation, faster training.
- 2. **Avoids overfitting:** Irrelevant features add noise; removing them improves generalisation.
- 3. **Improves accuracy:** Models trained on clean, relevant features perform better.
- 4. **Reduces storage:** Fewer features means less memory.
- 5. **Avoids the curse of dimensionality:** High-dimensional data is sparse and makes distance-based classifiers unreliable.
- 6. **Increases interpretability:** Fewer features make the model easier to understand.

Methods: Filter (ANOVA, Chi-square, mutual information), Wrapper (RFE), Embedded (LASSO).

Q1. (b)(i) One-Way ANOVA on Final Exam Score for Placement Prediction

Dataset:

Student	Study Hours	Final Exam Score	Placement
1	High	90	Y
2	Medium	85	N
3	Low	70	Y
4	High	95	Y
5	Medium	80	N
6	Low	65	N
7	High	92	Y
8	Medium	88	Y
9	Low	68	Y

ANOVA on FINAL EXAM SCORE (continuous) vs PLACEMENT (categorical: Y/N)

We test whether Final Exam Score differs significantly between Placement=Y and Placement=N.

Groups:

- Placement=Y: Students 1,3,4,7,8,9 → Scores: 90,70,95,92,88,68
- Placement=N: Students 2,5,6 → Scores: 85,80,65

Group Means: $\mu_Y = (90+70+95+92+88+68)/6 = 503/6 = 83.83$ $\mu_N = (85+80+65)/3 = 230/3 = 76.67$ Grand Mean $\mu = (503+230)/9 = 733/9 = 81.44$

SSB (Between groups): $SSB = n_Y(\mu_Y - \mu)^2 + n_N(\mu_N - \mu)^2 = 6(83.83 - 81.44)^2 + 3(76.67 - 81.44)^2 = 6(2.39)^2 + 3(-4.77)^2 = 6(5.712) + 3(22.753) = 34.27 + 68.26 = 102.53$

SSW (Within groups): Y deviations from 83.83: $(90-83.83)^2=38.07$, $(70-83.83)^2=191.27$, $(95-83.83)^2=124.75$, $(92-83.83)^2=66.75$, $(88-83.83)^2=17.39$, $(68-83.83)^2=250.63$ $SSW_Y = 38.07+191.27+124.75+66.75+17.39+250.63 = 688.86$

N deviations from 76.67: $(85-76.67)^2=69.39$, $(80-76.67)^2=11.09$, $(65-76.67)^2=136.19$ $SSW_N = 69.39+11.09+136.19 = 216.67$

$SSW = 688.86 + 216.67 = 905.53$

Degrees of Freedom: $df_{\text{between}} = k-1 = 2-1 = 1$ $df_{\text{within}} = n-k = 9-2 = 7$

Mean Squares: $MSB = 102.53/1 = 102.53$ $MSW = 905.53/7 = 129.36$

F-statistic: $F = MSB/MSW = 102.53/129.36 = 0.793$

Critical F at $\alpha=0.05$, $df_1=1$, $df_2=7$: From table = **5.59**

Conclusion: $F_{\text{calc}} (0.793) < F_{\text{critical}} (5.59) \rightarrow \text{Fail to reject } H_0 \therefore \text{Final Exam Score is NOT statistically significant for predicting Placement at 5\% level.}$

ANOVA on STUDY HOURS (categorical: High/Medium/Low) vs FINAL EXAM SCORE

We test if mean Final Exam Score differs across Study Hour groups.

Groups:

- High (1,4,7): 90, 95, 92 → $\mu_H = 277/3 = 92.33$
- Medium (2,5,8): 85, 80, 88 → $\mu_M = 253/3 = 84.33$
- Low (3,6,9): 70, 65, 68 → $\mu_L = 203/3 = 67.67$

Grand Mean = $733/9 = 81.44$

$$\text{SSB} = 3(92.33 - 81.44)^2 + 3(84.33 - 81.44)^2 + 3(67.67 - 81.44)^2 = 3(118.59) + 3(8.35) + 3(189.59) = 355.77 + 25.05 + 568.77 = \mathbf{949.59}$$

$$\text{SSW: High: } (90 - 92.33)^2 + (95 - 92.33)^2 + (92 - 92.33)^2 = 5.43 + 7.13 + 0.11 = 12.67$$

$$\text{Medium: } (85 - 84.33)^2 + (80 - 84.33)^2 + (88 - 84.33)^2 = 0.45 + 18.75 + 13.47 = 32.67$$

$$\text{Low: } (70 - 67.67)^2 + (65 - 67.67)^2 + (68 - 67.67)^2 = 5.43 + 7.13 + 0.11 = 12.67$$

$$\text{SSW} = 12.67 + 32.67 + 12.67 = \mathbf{58.01}$$

$$\text{df}_{\text{between}} = 3 - 1 = 2, \text{df}_{\text{within}} = 9 - 3 = 6$$

$$\text{MSB} = 949.59 / 2 = \mathbf{474.80} \quad \text{MSW} = 58.01 / 6 = \mathbf{9.67}$$

$$F = 474.80 / 9.67 = \mathbf{49.10}$$

Critical F at $\alpha=0.05$, $\text{df}_1=2$, $\text{df}_2=6$: From table = **5.14**

Conclusion: $F_{\text{calc}} (49.10) \gg F_{\text{critical}} (5.14) \rightarrow \text{Reject } H_0 \therefore \text{Study Hours per Week IS statistically significant for predicting Placement.}$

Feature Recommendation:

- **Study Hours per Week** $\rightarrow$ **RETAIN** ($F=49.10 \gg 5.14$, highly significant)
- **Final Exam Score** $\rightarrow$ **Can be dropped** ($F=0.793 < 5.59$, not significant for placement prediction)

Q1. (b)(ii) Null and Alternative Hypotheses

For Final Exam Score (ANOVA):

- H_0 : Mean Final Exam Score is equal across Placement groups (Y and N) $\rightarrow$ Score does not help predict placement.
- H_1 : Mean Final Exam Score differs significantly between Placement=Y and Placement=N.

For Study Hours (ANOVA):

- H_0 : $\mu_{\text{High}} = \mu_{\text{Medium}} = \mu_{\text{Low}}$ (Mean Final Exam Score is equal across all Study Hour groups)
- H_1 : At least one group mean (High/Medium/Low) differs significantly from the others.

Q2. (a) Differences Between ANOVA and Chi-Square in Feature Selection

Aspect	ANOVA	Chi-Square Test
Feature type	Continuous/numerical feature	Categorical/nominal feature
Target type	Categorical class label	Categorical class label

Aspect	ANOVA	Chi-Square Test
What it tests	Whether the mean of a continuous feature differs significantly across class groups	Whether two categorical variables are statistically independent
Null hypothesis	Group means are equal	Feature and target are independent
Test statistic	F-ratio = MSB/MSW	$\chi^2 = \sum (O-E)^2/E$
Assumption	Normal distribution within groups, equal variances	Sufficient expected frequency (≥ 5) in each cell
Example	Does salary (numerical) differ between placed/not-placed students?	Is city of residence (categorical) related to car ownership (categorical)?
Degrees of freedom	$df_1=k-1, df_2=n-k$	$df=(r-1)(c-1)$

Summary: Use **ANOVA** when the feature is continuous and the target is categorical. Use **Chi-Square** when both the feature and target are categorical.

Q2. (b)(i) Chi-Square Test: Car Ownership vs City of Residence

Dataset:

Person	City	Car
1	City A	Yes
2	City B	No
3	City C	Yes
4	City A	Yes
5	City B	Yes
6	City C	No
7	City A	No
8	City B	Yes
9	City C	Yes

Observed Contingency Table:

	Yes	No	Total
City A	2 (P1,P4)	1 (P7)	3
City B	2 (P5,P8)	1 (P2)	3
City C	2 (P3,P9)	1 (P6)	3
Total	6	3	9

Expected Values E = (Row Total × Col Total) / Grand Total:

$E(A, \text{Yes}) = 3 \times 6 / 9 = 2.0$; $E(A, \text{No}) = 3 \times 3 / 9 = 1.0$

$E(B, \text{Yes}) = 2.0$; $E(B, \text{No}) = 1.0$

$E(C, \text{Yes}) = 2.0$; $E(C, \text{No}) = 1.0$

Chi-Square: $\chi^2 = \sum (O - E)^2 / E$

$= (2-2)^2/2 + (1-1)^2/1 + (2-2)^2/2 + (1-1)^2/1 + (2-2)^2/2 + (1-1)^2/1 = 0 + 0 + 0 + 0 + 0 + 0 = \mathbf{0.00}$

df = $(3-1)(2-1) = 2$ **Critical χ^2** at $\alpha=0.05$, $df=2$: **5.99**

Conclusion: $\chi^2_{\text{calc}} (0.00) < \chi^2_{\text{critical}} (5.99) \rightarrow \text{Fail to reject } H_0 \therefore \text{Car Ownership is NOT dependent on City of Residence at 5\% significance level. The feature "City of Residence" should NOT be retained for predicting Car Ownership.}$

Q2. (b)(ii) Null and Alternative Hypotheses

- **H₀:** Car Ownership and City of Residence are **independent** of each other.
- **H₁:** Car Ownership and City of Residence are **dependent** (there is a significant association between them).

Q3. (a) Z-Score Standardisation

Z-Score Standardisation (Standard Score Normalisation) transforms each feature value by subtracting the mean and dividing by the standard deviation:

$$z = (x - \mu) / \sigma$$

where μ = mean of the feature, σ = standard deviation.

Changes after Z-score standardisation:

1. **Mean becomes 0:** Every feature is centred at zero.
2. **Standard deviation becomes 1:** All features have unit variance.

3. **Shape of distribution preserved:** The distribution's shape (skewness, kurtosis) does not change — only location and scale shift.
4. **Range changes:** No longer bounded to a fixed range (unlike min-max). Values typically lie in $[-3, +3]$.
5. **Units removed:** Features from different scales become comparable.
6. **Sensitive to outliers:** Since mean and std are affected by outliers, the z-scores of outliers become very large.

Q3. (b)(i) ANOVA: B.Tech Algorithm Marks vs School-Level Programming Knowledge

Dataset:

Roll	Knowledge	Marks
1	Good	80
2	Medium	85
3	Low	75
4	Good	60
5	Medium	75
6	Low	95
7	Good	70
8	Medium	87
9	Low	75

Groups:

- Good (1,4,7): 80,60,70 $\rightarrow \mu_G = 210/3 = 70.00$
- Medium (2,5,8): 85,75,87 $\rightarrow \mu_M = 247/3 = 82.33$
- Low (3,6,9): 75,95,75 $\rightarrow \mu_L = 245/3 = 81.67$

Grand Mean = $(210+247+245)/9 = 702/9 = 78.00$

SSB: $= 3(70-78)^2 + 3(82.33-78)^2 + 3(81.67-78)^2 = 3(64) + 3(18.77) + 3(13.47) = 192 + 56.31 + 40.41 = 288.72$

SSW: Good: $(80-70)^2 + (60-70)^2 + (70-70)^2 = 100+100+0 = 200$ Medium: $(85-82.33)^2 + (75-82.33)^2 + (87-82.33)^2 = 7.13+53.73+21.81 = 82.67$ Low: $(75-81.67)^2 + (95-81.67)^2 + (75-81.67)^2 =$

$$44.49 + 177.69 + 44.49 = 266.67 \text{ SSW} = 200 + 82.67 + 266.67 = \mathbf{549.34}$$

$$\text{df_between} = 2, \text{df_within} = 6$$

$$\text{MSB} = 288.72/2 = \mathbf{144.36} \text{ MSW} = 549.34/6 = \mathbf{91.56}$$

$$F = 144.36/91.56 = \mathbf{1.577}$$

Critical F at $\alpha=0.05$, $\text{df}_1=2$, $\text{df}_2=6$: **5.14**

Conclusion: $F_{\text{calc}} (1.577) < F_{\text{critical}} (5.14) \rightarrow \text{Fail to reject } H_0 \therefore \text{B.Tech Algorithm Marks is NOT significantly dependent on School-Level Programming Knowledge at 5\% significance.}$

Recommendation: Do NOT retain this feature for training.

Q3. (b)(ii) Null and Alternative Hypotheses

- $H_0: \mu_{\text{Good}} = \mu_{\text{Medium}} = \mu_{\text{Low}}$ (Mean Algorithm Marks is equal across all knowledge groups — the feature is not useful for prediction)
 - H_1 : At least one group mean differs from the others (knowledge level has a significant effect on marks)
-

Q4. (a) Importance of Degrees of Freedom in Chi-Square and ANOVA

Degrees of Freedom (df) represent the number of values in a statistical calculation that are free to vary — the amount of independent information available for estimating a parameter.

In Chi-Square Test: $\text{df} = (r-1)(c-1)$, where r = rows, c = columns in the contingency table.

- Determines which chi-square distribution to use for the critical value.
- Larger $\text{df} \rightarrow$ right-skewed distribution flattens $\rightarrow$ larger critical value needed for significance.
- Correctly computing df ensures we use the right reference distribution to avoid Type I errors.
- It reflects how many cells in the table are truly free once row/column totals are fixed.

In ANOVA: $\text{df_between} = k-1$ (k = number of groups) $\text{df_within} = n-k$ (n = total observations)

- df_between determines how many independent comparisons between group means are made.
- df_within measures the available variability within groups for estimating error.
- Both are needed to compute MSB and MSW correctly: $\text{MS} = \text{SS}/\text{df}$
- The F-distribution is parameterised by $(\text{df_between}, \text{df_within})$ — wrong df gives wrong critical value.

Q4. (b)(i) Chi-Square Test: Personal Vehicle vs Income Level

Dataset (12 persons):

Person	Income	Vehicle
1	High	Yes
2	Medium	Yes
3	Low	No
4	High	Yes
5	Medium	No
6	Low	Yes
7	High	Yes
8	Low	Yes
9	Medium	No
10	High	No
11	Low	No
12	High	Yes

Observed Table:

	Yes	No	Total
High	4 (1,4,7,12)	1 (10)	5
Medium	1 (2)	2 (5,9)	3
Low	2 (6,8)	2 (3,11)	4
Total	7	5	12

Expected Values: $E(H,Y)=5 \times 7/12=2.917$; $E(H,N)=5 \times 5/12=2.083$ $E(M,Y)=3 \times 7/12=1.750$;
 $E(M,N)=3 \times 5/12=1.250$ $E(L,Y)=4 \times 7/12=2.333$; $E(L,N)=4 \times 5/12=1.667$

Chi-Square: $\chi^2 = (4-2.917)^2/2.917 + (1-2.083)^2/2.083 + (1-1.750)^2/1.750 + (2-1.250)^2/1.250 +$
 $(2-2.333)^2/2.333 + (2-1.667)^2/1.667$

$= (1.083)^2/2.917 + (-1.083)^2/2.083 + (-0.750)^2/1.750 + (0.750)^2/1.250 + (-0.333)^2/2.333 +$
 $(0.333)^2/1.667$

$= 1.173/2.917 + 1.173/2.083 + 0.5625/1.750 + 0.5625/1.250 + 0.111/2.333 + 0.111/1.667$

$= 0.402 + 0.563 + 0.321 + 0.450 + 0.048 + 0.067$

$\chi^2 = 1.851$

$df = (3-1)(2-1) = 2$ **Critical χ^2 at $\alpha=0.05$, $df=2$: 5.99**

Conclusion: $\chi^2_{\text{calc}} (1.851) < \chi^2_{\text{critical}} (5.99) \rightarrow$ **Fail to reject H_0** $\therefore$ Personal Vehicle is **NOT**
statistically dependent on Income Level at 5% significance.

Q4. (b)(ii) Null and Alternative Hypotheses

- H_0 : Personal vehicle ownership and income level are **independent** of each other.
 - H_1 : Personal vehicle ownership and income level are **dependent** (there is a significant association).
-

Q5. (a) Derivation of LDA Discriminant Function

Goal: Find a linear projection w that maximises class separation.

Setup (Two-class case): Discriminant function: $g(x) = w^T x + w_0$

Fisher Criterion: Maximise: $J(w) = (w^T S_B w) / (w^T S_\phi w)$

Between-class scatter: $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$

Within-class scatter: $S_\phi = \Sigma_1 + \Sigma_2 = \sum_j \sum_{x_i \in C_j} (x_i - \mu_j)(x_i - \mu_j)^T$

Optimisation via Lagrange multiplier: Differentiate $J(w)$ w.r.t. w and set to zero: $\partial/\partial w [w^T S_B w - J \cdot w^T S_\phi w] = 0$
 $2 S_B w - 2J S_\phi w = 0$ $S_B w = J S_\phi w \rightarrow$ Generalised eigenvalue problem

Multiply both sides by S_ϕ^{-1} : $S_\phi^{-1} S_B w = J w$

Since $S_B w = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T w$ is in the direction of $(\mu_1 - \mu_2)$, the optimal projection is: $w = S_\phi^{-1}(\mu_1 - \mu_2)^*$

Discriminant function: $g(x) = w^T x - \text{threshold}$

Classification rule: Assign x to class 1 if $g(x) > \text{threshold}$ (midpoint in projected space), else class 2.

The **threshold** $w_0 = -w^T (\mu_1 + \mu_2)/2$ (midpoint of projected class means)

Final discriminant: $g(x) = (\mu_1 - \mu_2)^T S_\Phi^{-1} x - (1/2)(\mu_1 - \mu_2)^T S_\Phi^{-1}(\mu_1 + \mu_2)$

Q5. (b) Shared Assumptions of LDA and QDA; Differences

Shared Assumptions:

1. Features within each class follow a **Gaussian (Normal) distribution**.
2. Classes are **linearly separable** in the original feature space (for LDA) or in a quadratic space (QDA).
3. Training samples are **representative** of the true class distributions.
4. Features are **continuous**.

Differences:

Aspect	LDA	QDA
Covariance	Assumes equal covariance matrix Σ for all classes	Each class has its own covariance matrix Σ_j
Decision boundary	Linear (hyperplane)	Quadratic (curved surface)
Parameters	Fewer (shared Σ) $\rightarrow$ better with small data	More parameters $\rightarrow$ needs more data
Discriminant	$g(x) = w^T x + w_0$ (linear)	$g(x) = x^T A x + b^T x + c$ (quadratic)
Flexibility	Less flexible	More flexible
Risk	Underfitting if classes have very different covariances	Overfitting with small datasets

Q5. (c) Curse of Dimensionality in k-NN and Mitigation

How it affects k-NN:

1. In high dimensions, all pairwise distances converge to the same value $\rightarrow$ the concept of "nearest neighbour" becomes meaningless.
2. Data becomes exponentially sparse $\rightarrow$ a fixed-size neighbourhood captures fewer and fewer points.
3. Distance metrics (Euclidean) lose discriminative power — irrelevant dimensions dominate.
4. Performance degrades severely as d increases.

Mitigation techniques:

1. **Dimensionality reduction:** PCA, LDA, t-SNE before applying k-NN.
 2. **Feature selection:** Remove irrelevant features using ANOVA, chi-square, or mutual information.
 3. **Distance metric learning:** Use Mahalanobis distance (accounts for feature correlations) or learn a custom metric.
 4. **Approximate nearest neighbour:** Methods like KD-trees, ball trees, LSH (Locality Sensitive Hashing) for efficient search.
 5. **Increase k:** Larger k smooths the decision boundary, reducing the effect of sparse noise.
-

Q6. (a) 3-NN Classifier: Loan Prediction

(Identical to 2025 Q2(a) — full solution already provided. Summary:)

Test point: Age=32, Income=58K

Distances:

- (30,60): **2.83** → Yes ✓
- (35,55): **4.24** → Yes ✓
- (45,60): **13.15** → No

Majority (2 Yes vs 1 No): **Prediction = Yes (Loan Taken)**

Q6. (b) Minimum Distance Classifier

(Identical to 2025 Q2(b) — full solution already provided.)

Principle: Assign x to class $j^* = \operatorname{argmin} ||x - \mu_j||$

Distribution assumption: Spherical Gaussian with equal covariance and equal priors. Decision boundary is the perpendicular bisector of the line joining class means.

Q7. (a) Proof: High Dimensions → Near-Equidistance

(Identical to 2025 Q3(a) — full derivation already provided.)

Summary: By the Law of Large Numbers, $D(x_i, x_j)/\sqrt{d} \rightarrow \sigma\sqrt{2}$ for all pairs. The relative variance of distances $\rightarrow 0$ as $d \rightarrow \infty$, so all distances concentrate around the same value.

Q7. (b) Parametric vs Non-Parametric with Limited Data

(Identical to 2025 Q3(b) — full solution already provided.)

Answer: Parametric. Limited data + known distribution → use parametric (e.g., LDA, Gaussian NB) to exploit distributional knowledge and avoid overfitting.

Q7. (c) k-NN as Non-Parametric Learning

(Identical to 2025 Q3(c) — full solution already provided.)

Key points: No fixed model form, no training phase, model complexity grows with data, distribution-free, local decisions.

Q8. (a) LDA Projection — Full Calculation

Class ω_1 : (4,2),(2,4),(2,3),(3,6),(4,4) **Class ω_2 :** (9,10),(6,8),(9,5),(8,7),(10,8)

Step 1: Compute Class Means

$$\mu_1 = [(4+2+2+3+4)/5, (2+4+3+6+4)/5] = [15/5, 19/5] = (3.0, 3.8)$$

$$\mu_2 = [(9+6+9+8+10)/5, (10+8+5+7+8)/5] = [42/5, 38/5] = (8.4, 7.6)$$

Step 2: Compute Within-Class Scatter Matrices

$S_1 = \sum (x_i - \mu_1)(x_i - \mu_1)^T$ for ω_1 :

Deviations from $\mu_1 = (3.0, 3.8)$:

$$(4-3, 2-3.8) = (1, -1.8): \text{outer} = \begin{bmatrix} 1 & -1.8 \\ -1.8 & 3.24 \end{bmatrix}$$

$$(2-3, 4-3.8) = (-1, 0.2): \begin{bmatrix} 1 & -0.2 \\ -0.2 & 0.04 \end{bmatrix}$$

$$(2-3, 3-3.8) = (-1, -0.8): \begin{bmatrix} 1 & 0.8 \\ 0.8 & 0.64 \end{bmatrix}$$

$$(3-3, 6-3.8) = (0, 2.2): \begin{bmatrix} 0 & 0 \\ 0 & 4.84 \end{bmatrix}$$

$$(4-3, 4-3.8) = (1, 0.2): \begin{bmatrix} 1 & 0.2 \\ 0.2 & 0.04 \end{bmatrix}$$

$$S_1 = \begin{bmatrix} 1+1+1+0+1 & -1.8-0.2+0.8+0+0.2 \\ -1.8-0.2+0.8+0+0.2 & 3.24+0.04+0.64+4.84+0.04 \end{bmatrix} \\ = \begin{bmatrix} 4 & -1.0 \\ -1.0 & 8.80 \end{bmatrix}$$

$S_2 = \sum (x_i - \mu_2)(x_i - \mu_2)^T$ for ω_2 :

Deviations from $\mu_2 = (8.4, 7.6)$:

$$(9-8.4, 10-7.6) = (0.6, 2.4): \begin{bmatrix} 0.36 & 1.44 \\ 1.44 & 5.76 \end{bmatrix}$$

$$(6-8.4, 8-7.6) = (-2.4, 0.4): \begin{bmatrix} 5.76 & -0.96 \\ -0.96 & 0.16 \end{bmatrix}$$

$$(9-8.4, 5-7.6) = (0.6, -2.6): \begin{bmatrix} 0.36 & -1.56 \\ -1.56 & 6.76 \end{bmatrix}$$

$$(8-8.4, 7-7.6) = (-0.4, -0.6): \begin{bmatrix} 0.16 & 0.24 \\ 0.24 & 0.36 \end{bmatrix}$$

$$(10-8.4, 8-7.6) = (1.6, 0.4): \begin{bmatrix} 2.56 & 0.64 \\ 0.64 & 0.16 \end{bmatrix}$$

Step 3: Within-Class Scatter Matrix $S_\phi = S_1 + S_2$

$$S_\phi = [[4+9.20, -1.0-0.20], [-1.0-0.20, 8.80+13.20]] = [[13.20, -1.20], [-1.20, 22.00]]$$

Step 4: Between-Class Scatter Matrix S_B

$$\mu_1 - \mu_2 = [3.0 - 8.4, 3.8 - 7.6] = [-5.4, -3.8]$$

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T = [[29.16, 20.52], [20.52, 14.44]]$$

Step 5: Compute LDA Projection Vector $w = S_\phi^{-1}(\mu_1 - \mu_2)^$*

Inverse of S_ϕ : $\det(S_\phi) = (13.20)(22.00) - (-1.20)^2 = 290.40 - 1.44 = \mathbf{288.96}$

$$S_\phi^{-1} = (1/288.96) \times [[22.00, 1.20], [1.20, 13.20]] \\ = [[0.07614, 0.00415], [0.00415, 0.04567]]$$

$$w = S_\phi^{-1} \times (\mu_1 - \mu_2)^*:$$

$$w_1 = 0.07614(-5.4) + 0.00415(-3.8) = -0.4112 - 0.01577 = \mathbf{-0.4270} \quad w_2 = 0.00415(-5.4) + 0.04567(-3.8) = -0.02241 - 0.17355 = \mathbf{-0.1960}$$

LDA Projection Vector: $w = [-0.427, -0.196]^*$ (Or equivalently, in the direction **[0.427, 0.196]** or normalised: **[0.909, 0.418]**)

Projected means: $\tilde{y}_1 = w^T \mu_1 = (-0.427)(3.0) + (-0.196)(3.8) = -1.281 - 0.745 = \mathbf{-2.026}$ $\tilde{y}_2 = w^T \mu_2 = (-0.427)(8.4) + (-0.196)(7.6) = -3.587 - 1.490 = \mathbf{-5.077}$

Decision threshold $= (\tilde{y}_1 + \tilde{y}_2)/2 = (-2.026 - 5.077)/2 = \mathbf{-3.55}$

Classification: If $w^T x > -3.55 \rightarrow \omega_1$; else $\rightarrow \omega_2$.

Q8. (b) Limitations of LDA

1. **Assumes Gaussian distributions:** LDA performs poorly when class distributions are non-Gaussian (e.g., multimodal, skewed).
2. **Assumes equal covariance matrices:** If class covariances differ significantly, LDA makes suboptimal boundaries (QDA is better in that case).
3. **Linear boundaries only:** Cannot handle problems where classes are separated by non-linear boundaries.
4. **Requires $n > d$:** The within-class scatter matrix S_ϕ must be invertible; if $d > n$, S_ϕ is singular.

5. **At most $k-1$ discriminant directions:** For k classes, only $k-1$ meaningful projections exist, which may be too few for complex problems.
 6. **Sensitive to outliers:** Outliers distort the class means and covariance estimates.
-

Q9. (a) Decision Tree Root Node (GINI + Entropy)

(Identical to 2025 Q4(a) — complete solution already provided.)

Result: Both GINI and Entropy select **Pocket Money** as the root node.

- GINI Gain: Money=0.260 > Weather=0.176 > Humidity=0.000
 - Information Gain: Money=0.723 bits > Weather=0.496 bits > Humidity=0.281 bits
-

Q9. (b) Gini Impurity Function

(Identical to 2025 Q4(b).)

$$\text{Gini}(S) = 1 - \sum_j p(\omega_j)^2$$

Range: 0 (pure) to $(1-1/k)$ (maximum impurity). Used in CART. Computationally cheaper than entropy (no logarithms).

Q10. (a) Fundamental Assumption of Naïve Bayes

The fundamental assumption is **conditional independence** of features given the class label:

$$P(x_1, x_2, \dots, x_n | C) = P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$$

Each feature x_i is assumed to contribute independently to the class probability, with no correlation between features. This is "naïve" because features are almost always correlated in reality, but the classifier surprisingly works well despite this simplification.

Q10. (b) Naïve Bayes: Predict Purchase for (Female, Age>25, Salary=Medium)

Dataset (12 instances):

#	Gender	Age	Salary	Purchased
1	Male	≤25	Low	No
2	Female	>25	Low	No
3	Male	>25	Medium	Yes
4	Female	>25	High	Yes
5	Male	≤25	Medium	No
6	Female	>25	High	Yes
7	Male	≤25	High	Yes
8	Female	≤25	Medium	Yes
9	Male	>25	Low	No
10	Female	>25	High	Yes
11	Male	>25	Low	Yes
12	Female	≤25	Medium	No

Class counts: Yes=7 (3,4,6,7,8,10,11), No=5 (1,2,5,9,12) $P(\text{Yes})=7/12$, $P(\text{No})=5/12$

Compute $P(\text{feature}|\text{class})$:

$P(\text{Gender}=\text{Female}|\text{Yes})$: Female+Yes: 4,6,8,10 → $4/7$ **$P(\text{Gender}=\text{Female}|\text{No})$:** Female+No: 2,12 → $2/5$

$P(\text{Age}>25|\text{Yes})$: Age>25+Yes: 3,4,6,10,11 → $5/7$ **$P(\text{Age}>25|\text{No})$:** Age>25+No: 2,9 → $2/5$

$P(\text{Salary}=\text{Medium}|\text{Yes})$: Medium+Yes: 3,8 → $2/7$ **$P(\text{Salary}=\text{Medium}|\text{No})$:** Medium+No: 5,12 → $2/5$

Posterior for Yes: $P(\text{Yes}|X) \propto (7/12) \times (4/7) \times (5/7) \times (2/7) = (7 \times 4 \times 5 \times 2) / (12 \times 7 \times 7 \times 7) = 280/4116 = \mathbf{0.06803}$

Posterior for No: $P(\text{No}|X) \propto (5/12) \times (2/5) \times (2/5) \times (2/5) = (5 \times 2 \times 2 \times 2) / (12 \times 5 \times 5 \times 5) = 40/1500 = \mathbf{0.02667}$

Normalise: $P(\text{Yes}|X) = 0.06803 / (0.06803 + 0.02667) = 0.06803 / 0.09470 = \mathbf{0.718}$ $P(\text{No}|X) = \mathbf{0.282}$

Prediction: $P(\text{Yes}) > P(\text{No}) \rightarrow \text{The customer WILL Purchase.}$

Q10. (c) Handling Continuous Features in Naïve Bayes

For continuous features, we cannot use frequency counts. Two common approaches:

1. **Gaussian Naïve Bayes:** Assume the feature follows a Gaussian distribution within each class. Estimate μ_j and σ_j^2 from training data for class j : $P(x_i|C_j) = (1/\sqrt{2\pi\sigma_j^2}) \times \exp(-(x_i - \mu_j)^2 / (2\sigma_j^2))$
2. **Discretisation / Binning:** Convert continuous values into discrete bins (e.g., Low/Medium/High) and apply standard categorical Naïve Bayes.
3. **Kernel Density Estimation (KDE):** Non-parametric approach — estimate the density $P(x_i|C_j)$ using a kernel function (e.g., Gaussian kernel) fitted to training data.

Most common: Gaussian NB — simple, fast, works well when the Gaussian assumption holds.

Q11. (a) Bayes' Classification Error and Rule

Bayes' Classification Error: The Bayes error is the minimum possible misclassification error for a given problem — the irreducible error arising from class overlap in the feature space.

For a two-class problem:

$$P(\text{error}|x) = \min[P(C_1|x), P(C_2|x)]$$

$$\text{Overall Bayes error: } P(\text{error}) = \int \min[P(C_1|x)p(x), P(C_2|x)p(x)] dx$$

Deriving Bayes' Classification Rule: We want to minimise $P(\text{error})$. For a given x , assign to class C_j that maximises the posterior $P(C_j|x)$.

Proof: For two classes, $P(\text{error}|x) = 1 - \max[P(C_1|x), P(C_2|x)]$

This is minimised by choosing the class with the **maximum posterior probability**.

Bayes' Rule: Using Bayes' theorem: $P(C_j|x) = P(x|C_j)P(C_j) / P(x)$

Since $P(x)$ is the same for all classes, the rule becomes:

$$\text{Assign } x \text{ to class } j = \operatorname{argmax}_j [P(x|C_j) \times P(C_j)]^*$$

This is the **Maximum A Posteriori (MAP)** decision rule — the optimal Bayesian classifier that achieves the minimum possible error (Bayes error rate).

Decision boundary: $P(x|C_1)P(C_1) = P(x|C_2)P(C_2) \rightarrow$ **Likelihood ratio:** $P(x|C_1)/P(x|C_2) = P(C_2)/P(C_1)$

Q11. (b) Naïve Bayes: Predict Buys Computer

Predict: (age=youth, income=medium, student=yes, credit=fair) $\rightarrow$ buys computer?

P(feature|class):

P(age=youth|Yes): youth+Yes: 9,11 $\rightarrow 2/9$ **P(age=youth|No):** youth+No: 1,2,8 $\rightarrow 3/5$

P(income=medium|Yes): medium+Yes: 4,9,10,11,12 $\rightarrow 5$... let me recount. Medium records: 4(senior,medium,no,fair,yes), 8(youth,medium,no,fair,no), 10(senior,medium,yes,fair,yes), 11(youth,medium,yes,excellent,yes), 12(middle,medium,no,excellent,yes), 14(senior,medium,no,excellent,no) Medium+Yes: 4,10,11,12 $\rightarrow 4/9$ Medium+No: 8,14 $\rightarrow 2/5$

P(student=yes|Yes): student=yes+Yes: 5,7,9,10,11,13 $\rightarrow 6/9 = 2/3$ **P(student=yes|No):** student=yes+No: 6 $\rightarrow 1/5$

P(credit=fair|Yes): fair+Yes: 3,4,5,9,10,13 $\rightarrow 6/9 = 2/3$ **P(credit=fair|No):** fair+No: 1,8 $\rightarrow 2/5$

Posterior for Yes: $\propto (9/14) \times (2/9) \times (4/9) \times (6/9) \times (6/9) = (9 \times 2 \times 4 \times 6 \times 6) / (14 \times 9 \times 9 \times 9 \times 9) = 2592/91854 = \mathbf{0.02822}$

Posterior for No: $\propto (5/14) \times (3/5) \times (2/5) \times (1/5) \times (2/5) = (5 \times 3 \times 2 \times 1 \times 2) / (14 \times 5 \times 5 \times 5 \times 5) = 60/43750 = \mathbf{0.00137}$

Normalise: $P(\text{Yes}|X) = 0.02822 / (0.02822 + 0.00137) = 0.02822 / 0.02959 = \mathbf{0.954}$ $P(\text{No}|X) = \mathbf{0.046}$

Prediction: The person **WILL** buy a computer (Yes).

Q12. (a) MLP Weight Update — Sigmoid Activation

(Identical to 2025 Q5(a) — complete derivation already provided.)

Summary:

- Output layer: $\delta_j = (y - a_j) \cdot a_j(1 - a_j)$; $\mathbf{w}_{ij} \leftarrow \mathbf{w}_{ij} + \eta \cdot \delta_j \cdot \mathbf{a}_i$
 - Hidden layer: $\delta_i = a_i(1 - a_i) \cdot \sum_j (\delta_j \cdot \mathbf{w}_{ij})$; $\mathbf{v}_{ki} \leftarrow \mathbf{v}_{ki} + \eta \cdot \delta_i \cdot \mathbf{x}_k$
-

Q12. (b) SGD Prevents Overfitting

(Identical to 2025 Q5(b) — full solution already provided.)

Key reasons: noisy gradients act as implicit regulariser, escape sharp minima, natural early-stopping effect, breaks correlations in updates.

Q12. (c) Vanishing and Exploding Gradient

(Identical to 2025 Q5(c) — full solution already provided.)

Q13. (a) Activation Functions in ANN

Purpose of Activation Functions: Activation functions introduce **non-linearity** into the neural network, enabling it to learn and model complex, non-linear relationships between inputs and outputs. Without them, a multi-layer network would collapse to a single linear transformation regardless of depth.

Three Activation Functions:

1. Sigmoid: $f(x) = 1/(1+e^{-x})$; range: (0,1)

- Smooth, differentiable; ideal for binary output (probabilities)
- Derivative: $f'(x) = f(x)(1-f(x))$; max = 0.25
- Limitation: **Vanishing gradient** for large $|x|$; not zero-centred

2. Hyperbolic Tangent (tanh): $f(x) = (e^x - e^{-x})/(e^x + e^{-x})$; range: (-1,1)

- Zero-centred $\rightarrow$ better gradient flow than sigmoid
- Derivative: $f'(x) = 1-f(x)^2$; max = 1
- Still suffers from vanishing gradient for large $|x|$

3. ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$; range: $[0, \infty)$

- Computationally efficient; no vanishing gradient for $x > 0$
 - Derivative: $f'(x) = 1$ if $x > 0$, else 0
 - **Dead neuron** problem: neurons that output 0 may never activate again
 - Most widely used in modern deep learning
-

Q13. (b) Momentum in Backpropagation

Momentum is a technique that accelerates gradient descent by adding a fraction of the previous weight update to the current update, dampening oscillations and helping escape shallow local minima.

Standard SGD update: $\Delta w_{ij} = \eta \times \delta_j \times a_i$

SGD with Momentum: $v_t = \beta \times v_{t-1} + \eta \times \partial L / \partial w$ $w_t = w_{t-1} - v_t$

Where:

- $\beta \in [0,1)$ is the momentum coefficient (typically 0.9)
- v_t is the velocity (exponential moving average of past gradients)

Interpretation: Like a ball rolling downhill — it accumulates speed (momentum) in consistent directions and slows in oscillatory directions.

Benefits:

1. Faster convergence in flat regions (high curvature)
 2. Reduces oscillation in ravines of the loss surface
 3. Helps escape shallow local minima
 4. Effectively increases the learning rate in consistent gradient directions
-

Q13. (c) Batch vs Mini-Batch Gradient Descent

Aspect	Batch GD	Mini-Batch GD
Data used	Entire dataset per update	Small random subset (batch size b, typically 32-256)
Gradient	Exact gradient	Approximate gradient (less noisy than SGD)
Update frequency	Once per epoch	n/b times per epoch
Memory	Requires all data in memory	Moderate memory
Convergence	Smooth, stable but can be slow	Balanced — faster than batch, smoother than SGD
Parallelism	Can vectorise full dataset	Efficiently parallelised on GPU
Local minima	May get stuck	Noise helps escape local minima
Use case	Small datasets	Standard choice for deep learning

Key similarity: Both use gradient information to minimise the same loss function; both are deterministic given the same initial conditions (batch GD fully, mini-batch given same random seed).

Q14. (a) Softmax vs Hard-Max

Softmax: $P(\text{class } k \mid \mathbf{x}) = e^{z_k} / \sum_j e^{z_j}$

Hard-max (argmax): Output = 1 for $k^* = \text{argmax}_k z_k$, 0 for all others

Aspect	Softmax	Hard-Max
Output	Probability distribution (all sum to 1)	Binary one-hot vector
Differentiability	Fully differentiable	Not differentiable
Use	During training (backprop possible)	During inference (final decision)
Sensitivity	Amplifies largest logit, squashes others	Only the maximum survives
Uncertainty	Expresses confidence via probabilities	No uncertainty — winner-takes-all

Similarity: Both select the class with the largest score z_k as the predicted class. Both are monotonic with respect to z_k .

Q14. (b) Weight Update in Output Layer — Softmax + Categorical Cross-Entropy

Setup:

- Softmax output: $a_k = e^{(z_k)} / \sum_j e^{(z_j)}$ for class k
- Loss: Categorical Cross-Entropy: $L = -\sum_k y_k \log(a_k)$, where y_k is one-hot label

Gradient of Loss w.r.t. net input z_k : $\partial L / \partial z_k = a_k - y_k$

(This is the elegant result of combining softmax with cross-entropy: gradient = predicted – true)

Derivation: $\partial L / \partial z_k = \partial / \partial z_k [-\sum_j y_j \log(a_j)]$

Since $\partial a_j / \partial z_k = a_j (\delta_{jk} - a_k)$ (Jacobian of softmax):

$$= -\sum_j y_j \times (1/a_j) \times a_j (\delta_{jk} - a_k) = -\sum_j y_j (\delta_{jk} - a_k) = -y_k + a_k \sum_j y_j = -y_k + a_k \times 1 \text{ (since } \sum_j y_j = 1 \text{ for one-hot)} = a_k - y_k$$

Weight update in output layer: $\partial L / \partial w_{jk} = \partial L / \partial z_k \times \partial z_k / \partial w_{jk} = (a_k - y_k) \times a_j$ (hidden layer activation)

Update rule: $w_{jk}(\text{new}) = w_{jk}(\text{old}) - \eta (a_k - y_k) a_j$

where η = learning rate, a_j = activation from previous (hidden) layer, a_k = softmax output for class k , y_k = true label (one-hot).

Q15. (a) MLP as an Estimator — Steps

An MLP can function as a **function estimator** — learning any continuous mapping $f: \mathbb{R}^d \rightarrow \mathbb{R}$ (or $\mathbb{R}^m$) from training data.

Steps:

1. **Define architecture:** Choose number of layers L , neurons per layer, activation functions.
 - Hidden layers: ReLU/tanh for non-linearity
 - Output layer: Linear (for regression), sigmoid (binary), softmax (multiclass)
 2. **Initialise weights:** Use Xavier or He initialisation to prevent vanishing/exploding gradients.
 3. **Forward Pass:** For each training batch:
 - Input layer: $a^0 = x$
 - For each layer l : $net^l = W^l a^{l-1} + b^l$; $a^l = f(net^l)$
 - Output: $\hat{y} = a^L$
 4. **Compute Loss:** e.g., $MSE = (1/n) \sum (y_i - \hat{y}_i)^2$ for regression
 5. **Backward Pass (Backpropagation):**
 - Compute $\partial L / \partial w$ for each weight using chain rule
 - Output layer: $\delta^L = \partial L / \partial net = (\hat{y} - y) \cdot f'(net^L)$
 - Hidden layers: $\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'(net^l)$
 6. **Update Weights:** $w \leftarrow w - \eta \cdot \partial L / \partial w$
 7. **Repeat** steps 3–6 for multiple epochs until convergence.
 8. **Evaluate** on validation set; tune hyperparameters (η , layers, neurons).
 9. **Universal Approximation Theorem:** An MLP with one hidden layer and sufficient neurons can approximate any continuous function — the theoretical basis for MLP as estimator.
-

Q15. (b) Categorical Cross-Entropy Loss in Neural Networks

Categorical Cross-Entropy (CCE): For a K -class problem with one-hot labels y_k and softmax probabilities a_k :

$$L = -\sum_{k=1}^K y_k \log(a_k)$$

Since y_k is one-hot (only one $y_k=1$, rest=0):

$$L = -\log(a_{k^*}) \text{ where } k^* \text{ is the true class index}$$

Why used in Neural Networks:

1. **Penalises confident wrong predictions heavily:** If the true class has probability $a_{k^*} \rightarrow 0$, $loss \rightarrow \infty$. This creates strong gradients to correct severe misclassifications.
2. **Smooth gradient:** Combined with softmax, gradient $= (a_k - y_k)$, which is simple and numerically stable.
3. **Information-theoretic interpretation:** Minimising CCE is equivalent to maximum likelihood estimation of the softmax parameters.
4. **Multi-class support:** Naturally handles any number of classes.

Used in the output layer of classification networks (CNN, RNN, MLP) paired with softmax activation.

Q16. (a) Benefits of ReLU Activation

1. **No vanishing gradient:** For $x > 0$, ReLU derivative = 1 — gradient flows unchanged. Sigmoid/tanh saturate and produce near-zero gradients for large $|x|$.
2. **Computationally efficient:** $\max(0, x)$ is trivially fast; no exponentials needed.
3. **Sparsity:** Negative inputs $\rightarrow 0$ output. On average $\sim 50\%$ neurons inactive, making representations sparse and efficient.
4. **Faster convergence:** Empirically, networks with ReLU train $\sim 6\times$ faster than sigmoid networks.
5. **Reduces co-adaptation:** Sparse activation prevents neurons from over-relying on each other.

Limitation: Dead neuron problem (neurons stuck at 0). Addressed by Leaky ReLU: $f(x) = \max(0.01x, x)$.

Q16. (b) MLP Weight Update — ReLU Activation, Binary Classification

Setup:

- Activation: ReLU: $f(x) = \max(0, x)$; $f'(x) = 1$ if $x > 0$, else 0
- Loss: Binary Cross-Entropy: $L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$ (Or MSE: $L = (1/2)(y-\hat{y})^2$)
- Output neuron uses sigmoid (for probability); hidden neurons use ReLU

Output layer (sigmoid + BCE): $\partial L / \partial w_{ij} = (a_j - y_j) \times a_i$

where $a_j = \sigma(\text{net}_j)$ = output, a_i = ReLU hidden activation

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) - \eta(a_j - y_j)a_i$$

Hidden layer (ReLU): $\delta_i = [\sum_j \delta_j \cdot w_{ij}] \times f'(\text{net}_i)$

where $f'(\text{net}_i) = 1$ if $\text{net}_i > 0$, else 0 (ReLU derivative)

$$\partial L / \partial v_{ki} = \delta_i \times x_k$$

$$v_{ki}(\text{new}) = v_{ki}(\text{old}) - \eta \times \delta_i \times x_k$$

Key difference from sigmoid: The ReLU derivative is binary (0 or 1) — no saturation for positive inputs, but complete blocking (gradient=0) for negative inputs (dead neuron risk).

Q17. (a) K-Means Clustering (K=2, 2 Iterations)

Points:

Point	X	Y
p1	1	9
p2	2	10
p3	7	4
p4	10	3
p5	5	9
p6	7	2
p7	3	8
p8	4	10
p9	8	1
p10	9	3

Initial Seeds (randomly selected): C1=p1(1,9), C2=p4(10,3)

ITERATION 1: Assign to nearest centroid (Euclidean distance):

Point	d(C _i , C1=(1,9))	d(C _i , C2=(10,3))	Cluster
p1(1,9)	0	$\sqrt{(81+36)}$ = 10.82	C1
p2(2,10)	$\sqrt{(1+1)}$ = 1.41	$\sqrt{(64+49)}$ =10.63	C1
p3(7,4)	$\sqrt{(36+25)}$ = 7.81	$\sqrt{(9+1)}$ =3.16	C2
p4(10,3)	$\sqrt{(81+36)}$ =10.82	0	C2
p5(5,9)	$\sqrt{(16+0)}$ = 4.00	$\sqrt{(25+36)}$ =7.81	C1
p6(7,2)	$\sqrt{(36+49)}$ =9.22	$\sqrt{(9+1)}$ = 3.16	C2
p7(3,8)	$\sqrt{(4+1)}$ = 2.24	$\sqrt{(49+25)}$ =8.60	C1
p8(4,10)	$\sqrt{(9+1)}$ = 3.16	$\sqrt{(36+49)}$ =9.22	C1
p9(8,1)	$\sqrt{(49+64)}$ =10.63	$\sqrt{(4+4)}$ = 2.83	C2
p10(9,3)	$\sqrt{(64+36)}$ =10.00	$\sqrt{(1+0)}$ = 1.00	C2

$C1 = \{p1, p2, p5, p7, p8\}$, $C2 = \{p3, p4, p6, p9, p10\}$

New Centroids: C1: $((1+2+5+3+4)/5, (9+10+9+8+10)/5) = (15/5, 46/5) = (3.0, 9.2)$ C2: $((7+10+7+8+9)/5, (4+3+2+1+3)/5) = (41/5, 13/5) = (8.2, 2.6)$

ITERATION 2: Reassign with C1=(3.0,9.2), C2=(8.2,2.6):

Point	$d(C_i, C1=(3,9.2))$	$d(C_i, C2=(8.2,2.6))$	Cluster
p1(1,9)	$\sqrt{(4+0.04)}=2.01$	$\sqrt{(51.84+38.44)}=9.50$	C1
p2(2,10)	$\sqrt{(1+0.64)}=1.28$	$\sqrt{(38.44+54.76)}=9.65$	C1
p3(7,4)	$\sqrt{(16+27.04)}=6.56$	$\sqrt{(1.44+1.96)}=1.84$	C2
p4(10,3)	$\sqrt{(49+38.44)}=9.35$	$\sqrt{(3.24+0.16)}=1.84$	C2
p5(5,9)	$\sqrt{(4+0.04)}=2.01$	$\sqrt{(10.24+40.96)}=7.16$	C1
p6(7,2)	$\sqrt{(16+51.84)}=8.23$	$\sqrt{(1.44+0.36)}=1.34$	C2
p7(3,8)	$\sqrt{(0+1.44)}=1.20$	$\sqrt{(27.04+46.24)}=8.55$	C1
p8(4,10)	$\sqrt{(1+0.64)}=1.28$	$\sqrt{(17.64+54.76)}=8.51$	C1
p9(8,1)	$\sqrt{(25+67.24)}=9.60$	$\sqrt{(0.04+2.56)}=1.61$	C2
p10(9,3)	$\sqrt{(36+38.44)}=8.63$	$\sqrt{(0.64+0.16)}=0.89$	C2

$C1=\{p1, p2, p5, p7, p8\}$, $C2=\{p3, p4, p6, p9, p10\}$ (same as iteration 1 — converged!)

Final Clusters:

- **Cluster 1** (centroid 3.0, 9.2): p1(1,9), p2(2,10), p5(5,9), p7(3,8), p8(4,10)
- **Cluster 2** (centroid 8.2, 2.6): p3(7,4), p4(10,3), p6(7,2), p9(8,1), p10(9,3)

Q17. (b) Techniques for Choosing k in K-Means

1. **Elbow Method:** Plot WCSS (Within-Cluster Sum of Squares) vs k. The "elbow" point where the rate of decrease sharply drops is the optimal k. Beyond the elbow, adding more clusters gives diminishing returns.
2. **Silhouette Score:** For each k, compute the silhouette coefficient $s(i) = (b-a)/\max(a,b)$ for each point, where a=intra-cluster distance, b=nearest-cluster distance. Choose k with the highest average silhouette score (closer to 1 is better).

3. **Gap Statistic:** Compares the WCSS of the clustering to the expected WCSS under a null reference distribution. Choose k where the gap is maximised.
4. **Rule of Thumb:** $k \approx \sqrt{n/2}$ for n data points.
5. **Domain Knowledge:** If the application context suggests a specific number of groups (e.g., 3 product categories), use that.
6. **Information Criterion (BIC/AIC):** For model-based clustering (GMM), choose k minimising BIC or AIC.

Q18. (a) Agglomerative Hierarchical Clustering — Manhattan Distance

Points:

Point	X	Y
P1	1	9
P2	2	10
P3	3	4
P4	3	12
P5	4	9
P6	5	6

Manhattan Distance Matrix: $d(P_i,P_j) = |X_i-X_j| + |Y_i-Y_j|$

	P1	P2	P3	P4	P5	P6
P1	0	2	7	5	3	9
P2	2	0	9	3	5	9 → wait

Let me compute all: $d(P1,P2)=|1-2|+|9-10|=1+1=2$ $d(P1,P3)=|1-3|+|9-4|=2+5=7$
 $d(P1,P4)=|1-3|+|9-12|=2+3=5$ $d(P1,P5)=|1-4|+|9-9|=3+0=3$ $d(P1,P6)=|1-5|+|9-6|=4+3=7$
 $d(P2,P3)=|2-3|+|10-4|=1+6=7$ $d(P2,P4)=|2-3|+|10-12|=1+2=3$ $d(P2,P5)=|2-4|+|10-9|=2+1=3$
 $d(P2,P6)=|2-5|+|10-6|=3+4=7$ $d(P3,P4)=|3-3|+|4-12|=0+8=8$ $d(P3,P5)=|3-4|+|4-9|=1+5=6$
 $d(P3,P6)=|3-5|+|4-6|=2+2=4$ $d(P4,P5)=|3-4|+|12-9|=1+3=4$ $d(P4,P6)=|3-5|+|12-6|=2+6=8$
 $d(P5,P6)=|4-5|+|9-6|=1+3=4$

Distance Matrix:

	P1	P2	P3	P4	P5	P6
P1	[0, 2, 7, 5, 3, 7]					
P2	[2, 0, 7, 3, 3, 7]					
P3	[7, 7, 0, 8, 6, 4]					
P4	[5, 3, 8, 0, 4, 8]					
P5	[3, 3, 6, 4, 0, 4]					
P6	[7, 7, 4, 8, 4, 0]					

(i) Single Linkage (Minimum Distance)

Step 1: Minimum distance = $d(P1, P2) = 2 \rightarrow$ Merge $\{P1, P2\}$

Update using single linkage (min distance to remaining points):

- $\{P1, P2\}$ -P3: $\min(7, 7) = 7$
- $\{P1, P2\}$ -P4: $\min(5, 3) = 3$
- $\{P1, P2\}$ -P5: $\min(3, 3) = 3$
- $\{P1, P2\}$ -P6: $\min(7, 7) = 7$

Matrix:

	$\{P1, P2\}$	P3	P4	P5	P6
$\{P1, P2\}$	[0,	7,	3,	3,	7]
P3	[7,	0,	8,	6,	4]
P4	[3,	8,	0,	4,	8]
P5	[3,	6,	4,	0,	4]
P6	[7,	4,	8,	4,	0]

Step 2: Minimum = 3 (tie: $\{P1, P2\}$ -P4, $\{P1, P2\}$ -P5). Merge $\{P1, P2\}$ with P4 $\rightarrow \{P1, P2, P4\}$

Update:

- $\{P1, P2, P4\}$ -P3: $\min(7, 8) = 7$
- $\{P1, P2, P4\}$ -P5: $\min(3, 4) = 3$
- $\{P1, P2, P4\}$ -P6: $\min(7, 8) = 7$

Step 3: Minimum = 3 $\rightarrow$ Merge $\{P1, P2, P4\}$ -P5 $\rightarrow \{P1, P2, P4, P5\}$

Update:

- $\{P1, P2, P4, P5\}$ -P3: $\min(7, 6) = 6$
- $\{P1, P2, P4, P5\}$ -P6: $\min(7, 4) = 4$

Step 4: Minimum = $d(P3, P6) = 4 \rightarrow$ Merge $\{P3, P6\}$

Step 5: Minimum = 4 $\rightarrow$ Merge {P1,P2,P4,P5}-{P3,P6} $\rightarrow$ **All merged**

Single Linkage Dendrogram (merge heights):

```
Distance 2: P1 └─┐
                  │ {P1,P2}
Distance 2: P2 └─┐
                  │
                  │ (height 3)
Distance 3: P4 ───┴── {P1,P2,P4}
                  │ (height 3)
Distance 3: P5 ───┴── {P1,P2,P4,P5}
                  │
Distance 4: P3 └─┐
                  │ {P3,P6} ─────────┐
Distance 4: P6 └─┐ ─────────┴──
```

(ii) Complete Linkage (Maximum Distance)

Start with same distance matrix.

Step 1: Min distance = $d(P1,P2)=2 \rightarrow$ Merge {P1,P2}

Complete linkage uses MAX distance:

- {P1,P2}-P3: $\max(7,7)=7$
- {P1,P2}-P4: $\max(5,3)=5$
- {P1,P2}-P5: $\max(3,3)=3$
- {P1,P2}-P6: $\max(7,7)=7$

Matrix:

	{P1,P2}	P3	P4	P5	P6
{P1,P2}	[0, 7, 5, 3, 7]				
P3	[7, 0, 8, 6, 4]				
P4	[5, 8, 0, 4, 8]				
P5	[3, 6, 4, 0, 4]				
P6	[7, 4, 8, 4, 0]				

Step 2: Min = 3 ({P1,P2}-P5) $\rightarrow$ Merge $\rightarrow$ {P1,P2,P5}

Complete linkage:

- {P1,P2,P5}-P3: $\max(7,6)=7$
- {P1,P2,P5}-P4: $\max(5,4)=5$

- $\{P1, P2, P5\} - P6: \max(7, 4) = 7$

Step 3: $\text{Min} = d(P3, P6) = 4 \rightarrow \text{Merge } \{P3, P6\}$

Complete linkage:

- $\{P3, P6\} - \{P1, P2, P5\}: \max(7, 7) = 7$
- $\{P3, P6\} - P4: \max(8, 8) = 8$

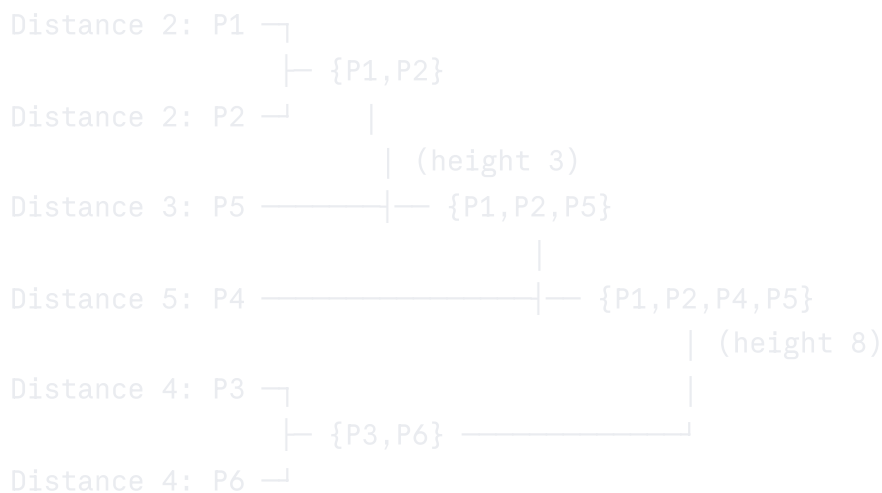
Step 4: $\text{Min} = 5 (\{P1, P2, P5\} - P4) \rightarrow \text{Merge} \rightarrow \{P1, P2, P4, P5\}$

Complete linkage:

- $\{P1, P2, P4, P5\} - \{P3, P6\}: \max(7, 8) = 8$

Step 5: Merge all $\rightarrow \{P1, P2, P3, P4, P5, P6\}$ at distance 8

Complete Linkage Dendrogram:



Q18. (b) Two Popular Distance Metrics in Clustering

1. Euclidean Distance: $d(p, q) = \sqrt{[\sum_i (p_i - q_i)^2]}$

- The "straight-line" distance in feature space.
- Sensitive to scale — features with larger range dominate.
- Works well for continuous, isotropic data.
- Standard choice for k-Means.

2. Manhattan Distance (L1 norm): $d(p, q) = \sum_i |p_i - q_i|$

- Sum of absolute differences along each dimension.
- Less sensitive to outliers than Euclidean (no squaring).
- Useful for high-dimensional data where L2 distances concentrate.
- Works well for grid-like or city-block geometries.

Q19. (a) K-Medoids Algorithm — 2 Iterations (Manhattan Distance)

(Identical to 2025 Q6(a) — complete solution already provided.)

Final Result after 2 Iterations:

- **Cluster 1** (Medoid R2=1.5,2.0): {R1(1,1), R2(1.5,2)}
 - **Cluster 2** (Medoid R5=3.5,5.0): {R3(3,4), R4(5,7), R5(3.5,5), R6(4.5,5), R7(3.5,4.5)}
-

Q19. (b) Major Drawbacks of K-Means

(Identical to 2025 Q6(b) — solution already provided.)

1. Must pre-specify k
 2. Sensitive to initial centroid selection
 3. Only finds spherical clusters
 4. Sensitive to outliers
 5. Assumes equal-sized clusters
-

Q20. (a) Agglomerative vs Divisive Hierarchical Clustering

Aspect	Agglomerative	Divisive
Direction	Bottom-up	Top-down
Start	Each point is its own cluster (n clusters)	All points in one cluster
Process	Iteratively MERGE the two closest clusters	Iteratively SPLIT the largest/most dissimilar cluster
End	One cluster containing all points	Each point in its own cluster
Complexity	$O(n^2 \log n)$ typical	$O(2^n)$ for optimal; $O(n^2)$ for heuristic
Common use	More widely used (AGNES algorithm)	Less common (DIANA algorithm)
Dendrogram	Read bottom-up — merges shown	Read top-down — splits shown

Q20. (b) EM Clustering vs K-Means

Aspect	EM Clustering	K-Means
Assignment	Soft (probabilistic membership)	Hard (binary assignment)
Cluster model	Gaussian (or other) distribution per cluster	No distributional assumption
Parameters	Learns mean, covariance, mixing weights	Learns only means
Cluster shape	Elliptical (via covariance matrix)	Spherical only
Update step	E-step + M-step (MLE update)	Assignment + centroid recalculation
Convergence	Guaranteed to increase log-likelihood	Guaranteed to decrease WCSS
Handling overlap	Natural (soft membership)	Poor (forced hard boundary)

Fundamental difference: EM is a probabilistic generative model that assigns partial membership, while K-means is a geometric algorithm that assigns crisp labels. EM is strictly more general — K-means is a special case of EM with hard assignments and spherical Gaussian clusters.

Q20. (c) Bisecting K-Means (Divisive)

Algorithm:

1. Start with all points in one cluster.
2. Select the cluster with the largest SSE (or largest size).
3. Apply standard k-means with $k=2$ to split it into two sub-clusters.
4. Of the two resulting splits, keep the one that minimises total SSE.
5. Repeat steps 2-4 until k clusters are obtained.

Advantages:

- More efficient than full divisive clustering ($O(kn)$ vs $O(2^n)$).
- Produces better global clusters than agglomerative (avoids local early-merge errors).
- Naturally hierarchical — all k values from 2 to n are available.

Limitations:

- The initial split decision affects all subsequent splits — errors propagate (no backtracking).
- Still requires specifying k .
- Repeated $k=2$ sub-problems may miss globally optimal cluster structures.

Q21. (a) Density-Based Reachability in DBSCAN

Key concepts:

ϵ -Neighbourhood: $N_{\epsilon}(p) = \{q \in D : \text{dist}(p,q) \leq \epsilon\}$ — all points within radius ϵ of p .

Core Point: p is a core point if $|N_{\epsilon}(p)| \geq \text{MinPts}$.

Directly Density-Reachable: q is directly density-reachable from p if:

- $q \in N_{\epsilon}(p)$ (q is in p 's neighbourhood), AND
- p is a core point

Density-Reachable: q is density-reachable from p if there exists a chain of points $p=p_1, p_2, \dots, p_n=q$ such that each p_{i+1} is directly density-reachable from p_i .

Density-Connected: Two points p and q are density-connected if there exists a point o from which both p and q are density-reachable.

A **cluster** in DBSCAN is defined as a maximal set of mutually density-connected points. Points not density-reachable from any core point are noise/outliers.

Q21. (b) DBSCAN on Given Points ($\epsilon=1$, $\text{MinPts}=3$, Manhattan distance)

Points: (3,0),(3,1),(3,2),(4,1),(4,2),(10,11),(10,10),(20,21),(20,22),(21,21),(21,22)

Manhattan Distance: $d = |\Delta x| + |\Delta y|$

Find ϵ -Neighbourhoods ($d \leq 1$) for each point:

(3,0): d to (3,1)=1 ✓, d to (3,2)=2 ✗, d to (4,1)=2 ✗ $\rightarrow N=\{(3,0),(3,1)\}; |N|=2 < 3 \rightarrow$ **Not core**

(3,1): d to (3,0)=1 ✓, d to (3,2)=1 ✓, d to (4,1)=1 ✓, d to (4,2)=2 ✗ $N=\{(3,1),(3,0),(3,2),(4,1)\}; |N|=4 \geq 3 \rightarrow$ **Core point**

(3,2): d to (3,1)=1 ✓, d to (4,2)=1 ✓, d to (4,1)=2 ✗, d to (3,0)=2 ✗ $N=\{(3,2),(3,1),(4,2)\}; |N|=3 \geq 3 \rightarrow$ **Core point**

(4,1): d to (3,1)=1 ✓, d to (4,2)=1 ✓, d to (3,2)=2 ✗, d to (3,0)=2 ✗ $N=\{(4,1),(3,1),(4,2)\}; |N|=3 \geq 3 \rightarrow$ **Core point**

(4,2): d to (3,2)=1 ✓, d to (4,1)=1 ✓, d to (3,1)=2 ✗ $N=\{(4,2),(3,2),(4,1)\}; |N|=3 \geq 3 \rightarrow$ **Core point**

(3,0): re-check: $N=\{(3,0),(3,1)\}; |N|=2 \rightarrow$ **Border point** (in neighbourhood of core (3,1))

(10,11): d to (10,10)=1 ✓, d to others $>> 1$ $N=\{(10,11),(10,10)\}; |N|=2 < 3 \rightarrow$ **Not core**

(10,10): d to (10,11)=1 ✓ $N=\{(10,10),(10,11)\}; |N|=2 < 3 \rightarrow$ **Not core**

(20,22): d to (20,21)=1 ✓ , d to (21,22)=1 ✓ , d to (21,21)=2 ✗ $N=\{(20,22),(20,21),(21,22)\}; |N|=3 \geq 3 \rightarrow$

Core point

(21,21): d to (20,21)=1 ✓ , d to (21,22)=1 ✓ , d to (20,22)=2 ✗ $N=\{(21,21),(20,21),(21,22)\}; |N|=3 \geq 3 \rightarrow$

Core point

(21,22): d to (20,22)=1 ✓ , d to (21,21)=1 ✓ , d to (20,21)=2 ✗ $N=\{(21,22),(20,22),(21,21)\}; |N|=3 \geq 3 \rightarrow$

Core point

Cluster Formation:

Cluster 1: Start from core (3,1). Density-reachable: (3,0),(3,1),(3,2),(4,1),(4,2) $\rightarrow$ **Cluster 1 = {(3,0), (3,1),(3,2),(4,1),(4,2)}**

Noise: (10,11),(10,10) — neither is a core point, and they are only reachable from each other (not from any core) $\rightarrow$ **Noise**

Cluster 2: Core points (20,21),(20,22),(21,21),(21,22) are all mutually density-connected. $\rightarrow$

Cluster 2 = {(20,21),(20,22),(21,21),(21,22)}

Final Result:

- **Cluster 1:** {(3,0),(3,1),(3,2),(4,1),(4,2)}
 - **Cluster 2:** {(20,21),(20,22),(21,21),(21,22)}
 - **Noise:** {(10,10),(10,11)}
-

Q22. (a) Advantages/Disadvantages of Density-Based Clustering + Hyperparameter Selection

Advantages:

1. Finds clusters of **arbitrary shape** — not limited to spherical clusters like k-Means.
2. **Automatically determines k** — no need to pre-specify number of clusters.
3. **Robust to outliers** — noise points are explicitly identified and excluded.
4. Works well with **spatial data** and geospatial applications.
5. No assumption about cluster distribution (non-parametric).

Disadvantages:

1. **Struggles with non-uniform density** — a single global ϵ and MinPts fails when clusters have varying densities.
2. **Sensitive to hyperparameters** — poor ϵ or MinPts leads to all points being noise or one giant cluster.
3. **High-dimensional data** — density estimates become unreliable (curse of dimensionality).

4. **Expensive for large datasets** — $O(n^2)$ naïve distance computations.
5. Border points assigned to first cluster that reaches them — non-deterministic.

Choosing Hyperparameters:

- **MinPts:** Rule of thumb: $\text{MinPts} \geq D+1$ (D = dimensions). Common choice: $\text{MinPts}=4$ or 5 .
 - **ϵ :** Use the **k-distance graph**: compute distance to k-th nearest neighbour for each point, sort descending, plot. Choose ϵ at the "elbow" of the curve.
-

Q22. (b) EM with Gaussian Mixture Model for Clustering

(Identical to 2024 Q7(b) — complete solution already provided.)

Summary: EM assumes data comes from K Gaussian components. Alternates between:

- **E-step:** Compute soft membership responsibilities r_{ik} using Bayes' theorem.
 - **M-step:** Update means μ_k , covariances Σ_k , mixing weights π_k using weighted MLE. Converges monotonically in log-likelihood.
-

Q23. (a) Fuzzy C-Means Algorithm

(Identical to 2025 Q7(a).)

Objective: Minimise $J_m = \sum_i \sum_j \mu_{ij}^m ||x_i - v_j||^2$

Steps: Initialise $U \rightarrow$ Compute centroids $v_j \rightarrow$ Update $\mu_{ij} \rightarrow$ Check convergence.

Q23. (b) FCM vs K-Means

(Identical to 2025 Q7(b).)

Key difference: K-Means $\rightarrow$ hard partition (binary membership). FCM $\rightarrow$ soft partition ($\mu_{ij} \in [0,1]$).

Q23. (c) Role of Fuzziness Parameter m

(Identical to 2025 Q7(c).)

$m \rightarrow 1$: crisp (like k-Means). $m=2$: typical balance. $m \rightarrow \infty$: all equal membership (useless). Typically $m \in [1.5, 3.0]$.

Given: Points P1(1,2), P2(2,3), P3(5,6); C=2, m=2

Initial: μ_1 : [0.8, 0.6, 0.1]; μ_2 : [0.2, 0.4, 0.9]

EPOCH 1:

Step 1: Compute Centroids:

$$v_1 = \sum_i \mu_{i1}^2 x_i / \sum_i \mu_{i1}^2$$

μ^2 values for C1: 0.64, 0.36, 0.01; Sum=1.01

μ^2 values for C2: 0.04, 0.16, 0.81; Sum=1.01

$$v_{1_x} = (0.64 \times 1 + 0.36 \times 2 + 0.01 \times 5) / 1.01 = (0.64 + 0.72 + 0.05) / 1.01 = 1.41 / 1.01 = \mathbf{1.396} \\ v_{1_y} = (0.64 \times 2 + 0.36 \times 3 + 0.01 \times 6) / 1.01 = (1.28 + 1.08 + 0.06) / 1.01 = 2.42 / 1.01 = \mathbf{2.396}$$

$$v_{2_x} = (0.04 \times 1 + 0.16 \times 2 + 0.81 \times 5) / 1.01 = (0.04 + 0.32 + 4.05) / 1.01 = 4.41 / 1.01 = \mathbf{4.366} \\ v_{2_y} = (0.04 \times 2 + 0.16 \times 3 + 0.81 \times 6) / 1.01 = (0.08 + 0.48 + 4.86) / 1.01 = 5.42 / 1.01 = \mathbf{5.366}$$

Step 2: Update Membership (m=2, so 2/(m-1)=2):

For each point: $\mu_{i1} = 1 / [1 + (d(x_i, v_1) / d(x_i, v_2))^2]$

Centroids: $v_1 = (1.396, 2.396)$, $v_2 = (4.366, 5.366)$

$$\mathbf{P1(1,2):} d_1 = \sqrt{[(1-1.396)^2 + (2-2.396)^2]} = \sqrt{[0.157 + 0.157]} = \sqrt{0.314} = 0.560 \\ d_2 = \sqrt{[(1-4.366)^2 + (2-5.366)^2]} = \sqrt{[11.33 + 11.33]} = \sqrt{22.66} = 4.760$$

$$\mu_{11} = 1 / [1 + (0.560 / 4.760)^2] = 1 / [1 + 0.01386] = 1 / 1.01386 = \mathbf{0.986} \quad \mu_{12} = 1 - 0.986 = \mathbf{0.014}$$

$$\mathbf{P2(2,3):} d_1 = \sqrt{[(2-1.396)^2 + (3-2.396)^2]} = \sqrt{[0.365 + 0.365]} = \sqrt{0.730} = 0.854 \\ d_2 = \sqrt{[(2-4.366)^2 + (3-5.366)^2]} = \sqrt{[5.597 + 5.597]} = \sqrt{11.19} = 3.345$$

$$\mu_{21} = 1 / [1 + (0.854 / 3.345)^2] = 1 / [1 + 0.0652] = 1 / 1.0652 = \mathbf{0.939} \quad \mu_{22} = \mathbf{0.061}$$

$$\mathbf{P3(5,6):} d_1 = \sqrt{[(5-1.396)^2 + (6-2.396)^2]} = \sqrt{[12.99 + 12.99]} = \sqrt{25.98} = 5.097 \\ d_2 = \sqrt{[(5-4.366)^2 + (6-5.366)^2]} = \sqrt{[0.402 + 0.402]} = \sqrt{0.804} = 0.897$$

$$\mu_{31} = 1 / [1 + (5.097 / 0.897)^2] = 1 / [1 + 32.26] = 1 / 33.26 = \mathbf{0.030} \quad \mu_{32} = \mathbf{0.970}$$

Updated Membership after Epoch 1:

Point	μ_1	μ_2
P1	0.986	0.014
P2	0.939	0.061
P3	0.030	0.970

EPOCH 2:

Recompute Centroids: μ^2 for C1: 0.972, 0.882, 0.0009; Sum=1.855 μ^2 for C2: 0.000196, 0.003721, 0.9409; Sum=0.944817

$$v_{1_x} = (0.972 \times 1 + 0.882 \times 2 + 0.0009 \times 5) / 1.855 = (0.972 + 1.764 + 0.0045) / 1.855 = 2.7405 / 1.855 = \mathbf{1.477}$$

$$v_{1_y} = (0.972 \times 2 + 0.882 \times 3 + 0.0009 \times 6) / 1.855 = (1.944 + 2.646 + 0.0054) / 1.855 = 4.5954 / 1.855 = \mathbf{2.478}$$

$$v_{2_x} = (0.000196 \times 1 + 0.003721 \times 2 + 0.9409 \times 5) / 0.9449 = (0.000196 + 0.007442 + 4.7045) / 0.9449 = 4.712 / 0.9449 = \mathbf{4.987}$$
$$v_{2_y} = (0.000196 \times 2 + 0.003721 \times 3 + 0.9409 \times 6) / 0.9449 = (0.000392 + 0.011163 + 5.6454) / 0.9449 = 5.657 / 0.9449 = \mathbf{5.988}$$

Updated membership for Epoch 2:

P1(1,2): $d_1 = \sqrt{[(1-1.477)^2 + (2-2.478)^2]} = \sqrt{[0.228 + 0.229]} = 0.676$; $d_2 = \sqrt{[(1-4.987)^2 + (2-5.988)^2]} = \sqrt{[15.90 + 15.90]} = 5.643$ $\mu_{11} = 1 / [1 + (0.676/5.643)^2] = 1 / [1 + 0.01432] = \mathbf{0.986}$; $\mu_{12} = \mathbf{0.014}$

P2(2,3): $d_1 = \sqrt{[(2-1.477)^2 + (3-2.478)^2]} = \sqrt{[0.274 + 0.272]} = 0.739$; $d_2 = \sqrt{[(2-4.987)^2 + (3-5.988)^2]} = \sqrt{[8.92 + 8.93]} = 4.232$ $\mu_{21} = 1 / [1 + (0.739/4.232)^2] = 1 / [1 + 0.0305] = \mathbf{0.970}$; $\mu_{22} = \mathbf{0.030}$

P3(5,6): $d_1 = \sqrt{[(5-1.477)^2 + (6-2.478)^2]} = \sqrt{[12.41 + 12.41]} = 4.982$; $d_2 = \sqrt{[(5-4.987)^2 + (6-5.988)^2]} = \sqrt{[0.000169 + 0.000144]} = 0.018$ $\mu_{31} = 1 / [1 + (4.982/0.018)^2] = 1 / [1 + 76634] \approx \mathbf{0.000013}$; $\mu_{32} \approx \mathbf{0.999987}$

Updated Membership after Epoch 2:

Point	μ_1	μ_2
P1	0.986	0.014
P2	0.970	0.030
P3	~0.000	~1.000

Interpretation: P1 and P2 strongly belong to Cluster 1; P3 very strongly belongs to Cluster 2.

Q24. (b) FCM vs K-Means: Handling Noise and Outliers

K-Means: Outliers severely distort the centroid since the mean is not robust to extreme values. An outlier far from the cluster will pull the centroid toward itself, degrading all membership assignments. K-Means has no concept of "borderline" — every point is forced into exactly one cluster.

FCM: Outliers receive low membership in all clusters but are still assigned partial membership everywhere. The fuzziness means their influence on centroids is proportional to μ_{ij}^m — since

$m > 1$, even a 0.5 membership contributes less than a 1.0 membership. This reduces (but doesn't eliminate) the effect of outliers.

However, FCM is still not robust to extreme outliers — they can distort centroids. For true noise robustness, **Possibilistic C-Means (PCM)** removes the constraint $\sum_j \mu_{ij} = 1$, allowing outliers to have low membership in ALL clusters simultaneously.

Q25. (a) Primary Goal of Dimension Reduction + Classifier Performance

(Identical to 2025 Q8(a) — complete solution already provided.)

Q25. (b) PCA and SVD Relationship

(Identical to 2025 Q8(b) — complete solution already provided.)

Summary: $X = U \Sigma V^T$. Right singular vectors V = PCA principal components. Eigenvalues $\lambda_i = \sigma_i^2 / (n-1)$. $Z = X \cdot V_k$ gives k -dimensional projection.

Q25. (c) Why PCA is Unsupervised

(Identical to 2025 Q8(c).)

PCA uses only X (no labels y), maximises variance irrespective of class structure, and is therefore unsupervised.

Q26. (a) Feature Selection vs Feature Extraction

Aspect	Feature Selection	Feature Extraction
Process	Selects a SUBSET of original features	Creates NEW features by transforming originals
Original features	Retained as-is	Replaced/combined
Interpretability	High — selected features remain meaningful	Lower — new features are combinations
Dimensionality	Reduced (subset of d)	Reduced ($k < d$ new features)
Method type	Filter, Wrapper, Embedded	PCA, LDA, ICA, Autoencoders

Aspect	Feature Selection	Feature Extraction
Example	Selecting "age" and "income" from 50 features	PCA creating 3 principal components from 50 features
Information	May discard useful correlated features	Captures variance from all features
When to use	When interpretability matters; when features are independent	When features are correlated; when all features carry some info

Q26. (b) Role of Eigenvalues and Eigenvectors in PCA

Eigenvectors of the covariance matrix define the **directions (principal components)** along which the data varies most.

Eigenvalues quantify the **amount of variance** explained in each eigenvector direction: λ_i = variance along the i-th principal component.

Relationship: $C \cdot w_i = \lambda_i \cdot w_i$ (Eigenvalue equation)

- The eigenvector with the **largest eigenvalue** = PC1 (direction of maximum variance)
- The eigenvector with the **second largest** = PC2 (orthogonal to PC1, next maximum variance)
- ... and so on

Determining Principal Components:

1. Compute $C = X^T X / (n-1)$
2. Find all eigenpairs (λ_i, w_i)
3. Sort by eigenvalue: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$
4. Select top k eigenvectors (those explaining e.g. 95% of total variance)
5. **Explained variance ratio** of PC_i = $\lambda_i / \sum_j \lambda_j$

Low eigenvalues → directions of little variance → usually noise → can be discarded.

Q26. (c) Why Data Must Be Mean-Centred Before PCA

Reason: PCA finds directions of maximum **variance**. Variance is measured relative to the mean. If data is not centred:

1. **The first principal component will point toward the data mean** (in the direction from origin to mean), not the direction of maximum spread. This is a spurious direction driven by data offset, not structure.

2. **Covariance matrix is incorrect:** The covariance formula $C = (1/n)X^T X$ assumes zero-mean data. For non-centred data, $X^T X$ computes the second moment, not the covariance.
3. **Mathematical consistency:** The eigenvectors of $X^T X$ and $(X - \mu)^T (X - \mu)$ are different. Only the centred version gives true principal components.

Process: $x_{\text{centred}} = x - \bar{\mu}$ (subtract feature-wise mean)

After centring, each feature has mean 0, and PCA correctly finds directions of maximum variance in the data cloud.

Q27. (a) Curse of Dimensionality + Dimension Reduction as Mitigation

Why it is problematic:

1. **Data sparsity:** Volume of space grows exponentially with dimensions — a fixed number of points becomes increasingly sparse.
2. **Distance concentration:** All pairwise distances converge to the same value $\rightarrow$ nearest-neighbour methods fail.
3. **Overfitting:** More features $\rightarrow$ more parameters $\rightarrow$ model memorises training data.
4. **Computational cost:** Training time and memory scale poorly with d .
5. **Sample complexity:** Exponentially more samples needed to maintain the same density.

How dimension reduction mitigates it:

1. **Reduces sparsity:** Projecting to lower-dimensional space restores meaningful density.
 2. **Restores distance discrimination:** In lower d , nearest neighbours are genuinely similar.
 3. **Removes noise dimensions:** Irrelevant features that add no signal are eliminated.
 4. **Reduces overfitting risk:** Fewer dimensions $\rightarrow$ simpler model $\rightarrow$ better generalisation.
 5. PCA keeps directions of maximum variance; LDA keeps class-discriminative directions.
-

Q27. (b) Primary Objective of PCA + Feature Transformation

Primary Objective: PCA transforms the original d -dimensional correlated feature space into a new k -dimensional ($k < d$) uncorrelated feature space, such that **maximum variance is preserved** in the reduced representation.

How it transforms features:

1. **Mean-centre** the data: $x \leftarrow x - \mu$
2. **Compute covariance** matrix $C = X^T X / (n-1)$
3. Find **eigenvectors** (principal directions) and **eigenvalues** (variances) of C
4. **Sort** eigenvectors by eigenvalue (descending)
5. **Project** data: $Z = X \cdot W_k$ (where W_k = top k eigenvectors)

Result:

- New features (principal components) are **linear combinations** of original features
 - They are **mutually orthogonal** (uncorrelated)
 - Each explains a **decreasing amount of variance**
 - Original correlated features → new uncorrelated, compressed representation
-

Q28. (a) Explained Variance Ratio in PCA

The **explained variance ratio** of the i -th principal component is: $\text{EVR}_i = \lambda_i / \sum_{j=1}^d \lambda_j$

It represents the **fraction of the total dataset variance captured** by the i -th principal component.

Interpretation:

- If $\text{EVR}_1 = 0.72$, PC1 alone captures 72% of all information in the dataset.
- Cumulative $\text{EVR} = \sum_{i=1}^k \text{EVR}_i$ tells how much total variance is retained by keeping k components.
- Standard practice: choose k such that cumulative $\text{EVR} \geq 0.95$ (95% variance retained).

Practical use: Plotting cumulative EVR vs number of components (scree plot) helps choose the optimal k — select k at the "elbow" or where cumulative EVR crosses the 95% threshold.

Q28. (b) How PCA Handles Correlated Features

PCA naturally handles correlated features through the **covariance matrix**:

1. If features are correlated, the off-diagonal elements of C are non-zero.
2. The eigenvectors of C align with the natural axes of the data ellipsoid — including oblique axes defined by correlations.
3. The principal components are **linear combinations** of the original features that capture the shared variance from correlations.
4. Specifically, strongly correlated features will have large off-diagonal covariance entries, causing PCA to combine them into a single principal component (capturing their shared direction).

Result: Correlated features are "merged" into a smaller number of principal components. The dimensionality reduction is most effective when features are strongly correlated (more redundancy to remove).

Q28. (c) Can ICA Uniquely Determine Original Sources?

Yes, under specific conditions — with one key exception.

ICA can uniquely recover sources when:

1. At most one source is Gaussian (the rest must be non-Gaussian).
2. Sources are statistically independent.
3. The mixing is a square, invertible linear matrix.

Uniqueness holds up to:

- **Permutation:** The order of recovered sources may differ from the original order.
- **Scaling:** Each source can only be recovered up to a scalar factor (sign and amplitude may differ).

Why ICA fails with Gaussian sources: A multivariate Gaussian distribution is rotationally symmetric — any rotation of independent Gaussian sources produces another set of Gaussian signals with identical statistical properties. Therefore, the mixing matrix cannot be uniquely identified from Gaussian data.

Conclusion: ICA can **uniquely** determine original sources (up to permutation and scale) if the sources are non-Gaussian. For Gaussian sources, unique recovery is impossible.

Q29. (a) Key Assumptions of ICA About Source Signals

1. **Statistical independence:** Source signals s_i and s_j ($i \neq j$) are statistically independent:
 $P(s_1, \dots, s_n) = \prod_i P(s_i)$.
 2. **Non-Gaussianity:** At most one source can be Gaussian. The non-Gaussianity is what ICA exploits to separate sources (measured via kurtosis or negentropy).
 3. **Linear mixing:** Observed signals $x = A \cdot s$ (matrix multiplication). The mixing is a linear instantaneous transformation.
 4. **Square mixing matrix (basic ICA):** Same number of observations as sources ($n = m$), and A is invertible, so $W = A^{-1}$ exists.
 5. **No noise** (basic ICA): Observed signals are pure mixtures without additive noise.
-

Q29. (b) Why ICA Fails with Gaussian Sources

Mathematical reason: The probability density of a multivariate Gaussian is: $P(x) \propto \exp(-(1/2)x^T \Sigma^{-1}x)$

If sources s are i.i.d. Gaussian with identity covariance, and $x = As$:

$$P(x) \propto \exp(-(1/2)x^T (AA^T)^{-1}x)$$

Consequence: The likelihood function is flat over all rotations of A — gradient-based optimisation cannot determine the correct A . ICA algorithms (which maximise non-Gaussianity/independence) find no gradient signal in purely Gaussian data.

Q29. (c) Why ICA is a "Blind Source Separation" Technique

Blind Source Separation (BSS) refers to recovering original source signals without knowledge of:

1. The mixing matrix A (how signals were combined)
2. The sources themselves

ICA is "blind" in the sense that:

1. **No prior knowledge of sources:** We only observe mixtures $x = As$. Neither the sources s nor A are known.
2. **No training labels:** ICA is completely unsupervised — it uses only the statistical properties of the observed data.
3. **Assumptions are minimal:** Only statistical independence and non-Gaussianity of sources are assumed — these are very general conditions.

Classical example — Cocktail party problem: Multiple microphones record overlapping speech signals. ICA separates each person's voice (source) from the mixture without knowing what anyone said or where the microphones were placed.

Q30. (a) Primary Working Principle of ICA in Dimension Reduction

ICA (Independent Component Analysis) finds a linear transformation W such that the transformed components $y = Wx$ are as statistically independent as possible.

Working Principle:

1. **Model:** Observed data $x = As$, where A is the unknown mixing matrix and s are independent sources.
2. **Goal:** Find $W = A^{-1}$ such that $y = Wx \approx s$ (recover independent sources).
3. **Independence measure:** Use non-Gaussianity as a proxy for independence (Central Limit Theorem: mixtures of independent signals become more Gaussian; we want the opposite).
4. **Objective:** Maximise non-Gaussianity of each component (using kurtosis or negentropy).
5. **Algorithm (FastICA):** Iteratively update W using a fixed-point algorithm until components are maximally non-Gaussian/independent.

For dimension reduction: Keep only the k most non-Gaussian independent components, discarding Gaussian ones (which likely represent noise).

Q30. (b) ICA vs PCA: Objective Difference

Aspect	PCA	ICA
Objective	Maximise variance of projections	Maximise statistical independence of components
Criterion	Second-order statistics (covariance)	Higher-order statistics (non-Gaussianity)
Components	Orthogonal (uncorrelated)	Independent (stronger condition than uncorrelated)
Result	Directions of maximum variance	Directions of maximum non-Gaussianity
Use case	Compression, noise reduction, visualisation	Blind source separation, feature discovery
Handles non-linearity	No	No (linear ICA)
Supervised	No	No
Gaussian data	Works well	Fails (cannot distinguish rotations)
Uniqueness	Up to scaling (eigenvalues may be equal)	Up to permutation and scaling (for non-Gaussian sources)

Core difference: PCA finds the axes of greatest data spread (variance). ICA finds the axes of greatest statistical independence — these are different unless data happens to be Gaussian (where the two coincide).

Q31. (a) CNN Feature Extraction

(Identical to 2025 Q9(a) — complete solution already provided.)

Q31. (b) LDA for Dimension Reduction

(Identical to 2025 Q9(b) — complete solution already provided.)

Q32. (a) LDA vs PCA in Dimension Reduction

Aspect	PCA	LDA
Type	Unsupervised	Supervised
Criterion	Maximises total variance	Maximises class separability (Fisher criterion)
Uses labels	No	Yes
Max components	$\min(n-1, d)$	$k-1$ (k = number of classes)
Components	Orthogonal eigenvectors of covariance matrix	Eigenvectors of $S_{\phi}^{-1}S_B$
Best for	Visualisation, compression	Classification preprocessing
Handles overlap	Ignores class structure	Explicitly separates classes
Example	Compress 1000 features to 50	Compress 1000 features to 9 for 10-class problem

Key insight: PCA is blind to class labels and may keep components that are useless for classification. LDA explicitly optimises for class discrimination — it's almost always better for classification preprocessing when labels are available.

Q32. (b) Kernel PCA and Its Advantages

Kernel PCA extends standard PCA to non-linear dimension reduction using the **kernel trick**.

Idea: Map data to a high-dimensional feature space $\phi: \mathbb{R}^d \rightarrow \mathcal{F}$ (implicitly, via a kernel function), then apply PCA in $\mathcal{F}$.

Kernel trick: Instead of computing $\phi(x_i) \cdot \phi(x_j)$ explicitly, compute $k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$. Common kernels: RBF: $k(x, y) = \exp(-||x - y||^2 / 2\sigma^2)$, Polynomial: $k(x, y) = (x^T y + c)^d$

Algorithm:

1. Compute $n \times n$ kernel matrix K where $K_{ij} = k(x_i, x_j)$
2. Centre K in feature space
3. Find top k eigenvectors of K
4. Project data onto these eigenvectors

Advantages over standard PCA:

1. **Handles non-linearly separable data:** Can find non-linear principal components that capture complex manifold structures invisible to linear PCA.
2. **More expressive:** With RBF kernel, maps to infinite-dimensional space — can separate any configuration of data (in principle).
3. **No explicit computation of ϕ :** The kernel trick makes it computationally tractable.
4. **Better classification preprocessing:** When classes are non-linearly distributed, Kernel PCA reveals structure that standard PCA misses.

Limitation: $O(n^2)$ memory for kernel matrix; not suitable for very large datasets.

Q33. (a) Principal Component Vectors — Step by Step

Data: | X | 2 | 3 | 7 | | Y | 11 | 14 | 26 |

Step 1: Compute Means: $\mu_X = (2+3+7)/3 = 12/3 = 4$ $\mu_Y = (11+14+26)/3 = 51/3 = 17$

Step 2: Mean-centre the data:

	X	Y
Original	(2,11)	(3,14)
Centred	(-2,-6)	(-1,-3)

Step 3: Compute Covariance Matrix: $C = (1/(n-1)) \times \sum (x_i - \mu)(x_i - \mu)^T$ (using $n-1=2$)

$$C_{11} = [(-2)^2 + (-1)^2 + (3)^2]/2 = [4+1+9]/2 = 14/2 = 7 \quad C_{12} = C_{21} = [(-2)(-6) + (-1)(-3) + (3)(9)]/2 = [12+3+27]/2 = 42/2 = 21 \quad C_{22} = [(-6)^2 + (-3)^2 + (9)^2]/2 = [36+9+81]/2 = 126/2 = 63$$

Covariance Matrix: $C = [[7, 21], [21, 63]]$

Step 4: Find Eigenvalues: $\det(C - \lambda I) = 0$

$$(7-\lambda)(63-\lambda) - 21^2 = 0$$

$$441 - 70\lambda + \lambda^2 - 441 = 0$$

$$\lambda^2 - 70\lambda = 0$$

$$\lambda(\lambda - 70) = 0$$

Eigenvalues: $\lambda_1 = 70, \lambda_2 = 0$

($\lambda_2=0$ because the 3 data points are collinear — they lie on a perfect straight line!)

Step 5: Find Eigenvectors:

For $\lambda_1 = 70$: $(C - 70I)v = 0$ $[[7-70, 21], [21, 63-70]]v = 0$ $[[-63, 21], [21, -7]]v = 0$

From row 1: $-63v_1 + 21v_2 = 0 \rightarrow v_2 = 3v_1$

Eigenvector: $v_1 = [1, 3]^T$

Normalised: $\|v_1\| = \sqrt{1+9} = \sqrt{10}$ **PC1** = $[1/\sqrt{10}, 3/\sqrt{10}] = [0.316, 0.949]$

For $\lambda_2 = 0$: $(C - 0 \cdot I)v = 0$ $[[7, 21], [21, 63]]v = 0$

$7v_1 + 21v_2 = 0 \rightarrow v_1 = -3v_2$ Eigenvector: $v_2 = [3, -1]^T$ (orthogonal to PC1) Normalised: **PC2** = $[3/\sqrt{10}, -1/\sqrt{10}] = [0.949, -0.316]$

Step 6: First Principal Component Projection:

Project centred data onto PC1 = $[0.316, 0.949]$:

$z_1 = [-2, -6] \cdot [0.316, 0.949] = -0.632 - 5.694 = -6.326$ $z_2 = [-1, -3] \cdot [0.316, 0.949] = -0.316 - 2.847 = -3.163$ $z_3 = [3, 9] \cdot [0.316, 0.949] = 0.948 + 8.541 = 9.489$

First PC captures 70/70 = 100% of variance (since $\lambda_2=0$, no information in the second direction).

Summary:

- Covariance Matrix: $C = [[7, 21], [21, 63]]$
 - PC1 ($\lambda=70$, 100% variance): $w_1 = [0.316, 0.949]$
 - PC2 ($\lambda=0$, 0% variance): $w_2 = [0.949, -0.316]$
 - Projections on PC1: $[-6.326, -3.163, 9.489]$
-

Q33. (b) Two Advantages and Two Disadvantages of PCA

Advantages:

1. **Dimensionality reduction without label information:** PCA is unsupervised and removes redundancy by finding directions of maximum variance. It can dramatically reduce storage and computation — a 1000-feature dataset might be compressible to 20 principal components retaining 95% of variance.
2. **Removes correlation (decorrelation):** PCA outputs uncorrelated principal components. Many algorithms (e.g., linear regression, Gaussian NB) work better or faster with uncorrelated features. This also removes multicollinearity.

Disadvantages:

1. **Ignores class structure:** Being unsupervised, PCA may discard dimensions with low variance that are highly discriminative for classification. A direction orthogonal to the PC1 might be the most important for separating two classes — PCA would discard it.
2. **Linear transformations only:** Standard PCA can only capture linear relationships. Non-linear manifolds (Swiss roll, circular clusters) are poorly represented. Kernel PCA addresses this but at increased cost. Also, principal components are linear combinations of original features — they lose the interpretability of individual features.

END OF ALL SUGGESTION QUESTION SOLUTIONS

Total: 33 Questions, All Solved
